



Chapter 1

Basics of Software Testing

WHY SOFTWARE TESTING?

- 💡 Testing is important because software bugs could be expensive or even dangerous.
- 💡 Software bugs can potentially cause monetary and human loss
- 💡 It's essential since it makes sure that the customer finds the organization reliable and their satisfaction in the application is maintained.
- 💡 It is very important to ensure the Quality of the product.

SOFTWARE TESTING(ST)

💡 ST is a process to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.

Basic Definitions: Errors:



- ✚ **Errors** are a part of our daily life.
- ✚ Humans make errors in their thoughts, actions, and in the products that might result from their actions.
- ✚ Errors occur wherever humans are involved in taking actions and making decisions.



💡 A synonym for error is a mistake.

💡 When people make mistakes while coding , they are called bugs.

💡 An error may be magnified during design and amplified still more during coding.

Fault:



- 💡 A fault is the result of an error.
- 💡 Defect is a good synonym for fault.
- 💡 2 types of faults.
 - ✓ Fault of omission
 - ✓ Fault of commission



- 🧠 Fault of omission occurs when we fail to enter correct information.
- 🧠 Fault of commission occurs when we enter something into a representation that is incorrect.
- 🧠 Of the two types faults of omission are more difficult to detect and resolve.

Failure:



- 💡 A failure occurs when a fault executes.
- 💡 Failures only occur in an executable representation, which is usually taken to be source code, or more precisely, loaded object code.

Incident:



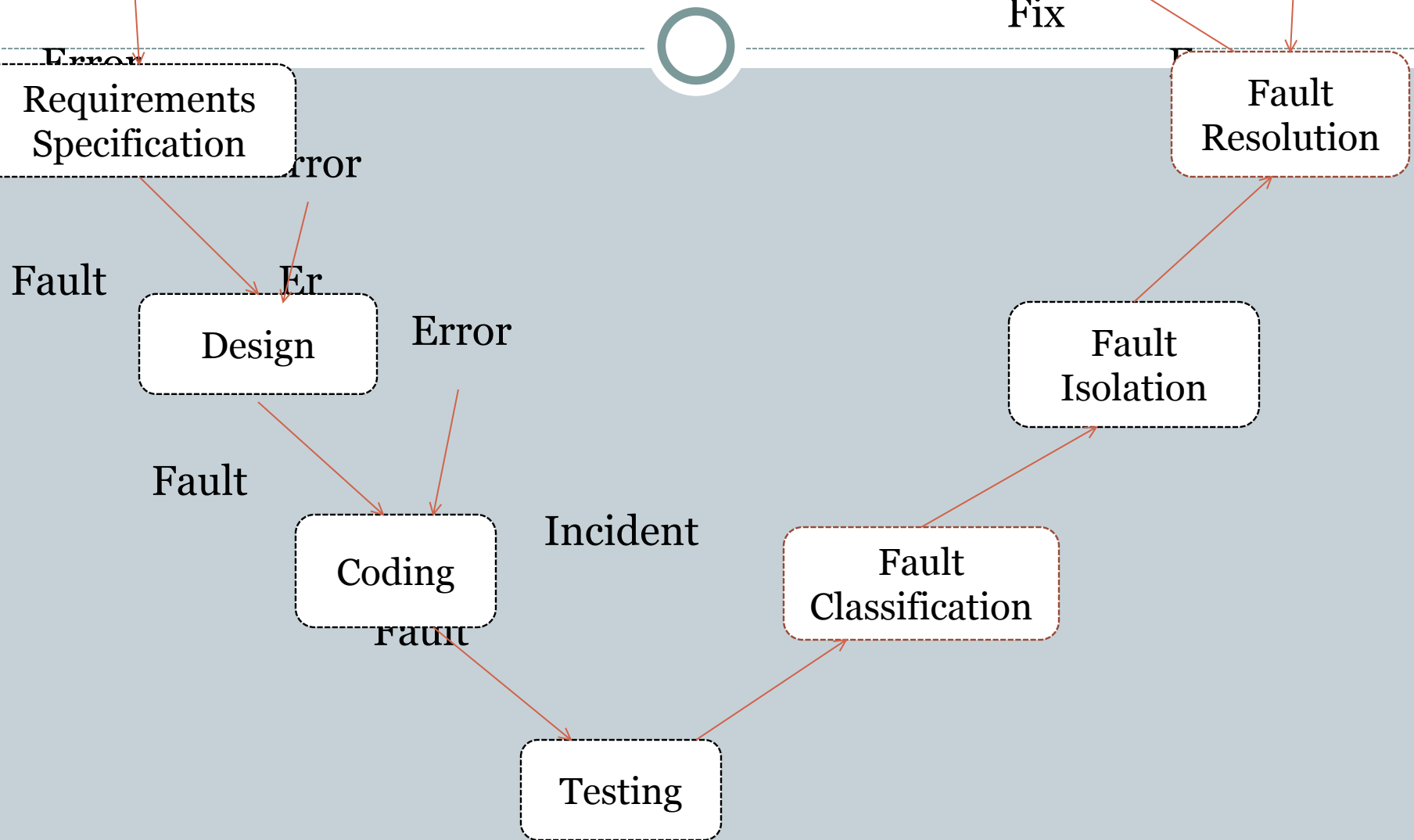
- 💡 Whenever a failure occurs, it may or may not be readily visible to the user.
- 💡 An incident is the symptom associated with a failure that alerts the user to the occurrence of a failure.

Test:



- 💡 Testing is concerned with errors, faults, failures and incidents.
- 💡 A test is exercising software using test cases.
- 💡 A test has 2 distinct goals.
 - ✓ To find failures.
 - ✓ To demonstrate correct execution.

A Testing Life Cycle



💡 The following points can be summarized from the testing life cycle.



💡 The first three phases are putting bugs in.

💡 The testing phase is finding bugs.

💡 The last three phases are getting bugs out.

💡 The fault resolution is another opportunity for errors.



 The process of testing can be subdivided into separate steps:

- ✓ Test planning
- ✓ Test case development
- ✓ Running test cases
- ✓ Evaluating test results.

Test Cases:



- 💡 A TEST CASE is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly.
- 💡 What information should be in a test case?
 - ✓ Inputs : Preconditions and actual inputs.
 - ✓ Expected outputs : Post conditions and actual outputs.

Test Case Example / Test Case Sample

Test Suite ID TS001

Test Case ID TC001

Test Case Summary To verify that clicking the Generate Coin button generates coins.

Related Requirement RS001

Prerequisites User is authorized. Coin balance is available.

Test Procedure Select the coin denomination in the Denomination field.
Enter the number of coins in the Quantity field.
Click Generate Coin.

Test Data Denominations: 0.05, 0.10, 0.25, 0.50, 1, 2, 5

Quantities: 0, 1, 5, 10, 20

Expected Result Coin of the specified denomination should be produced if the specified Quantity is valid (1, 5)
A message '_Please enter a valid quantity between 1 and 10' should be displayed if the specified quantity is invalid.

Actual Result If the specified quantity is valid, the result is as expected.
If the specified quantity is invalid, nothing happens; the expected message is not displayed

Status Fail

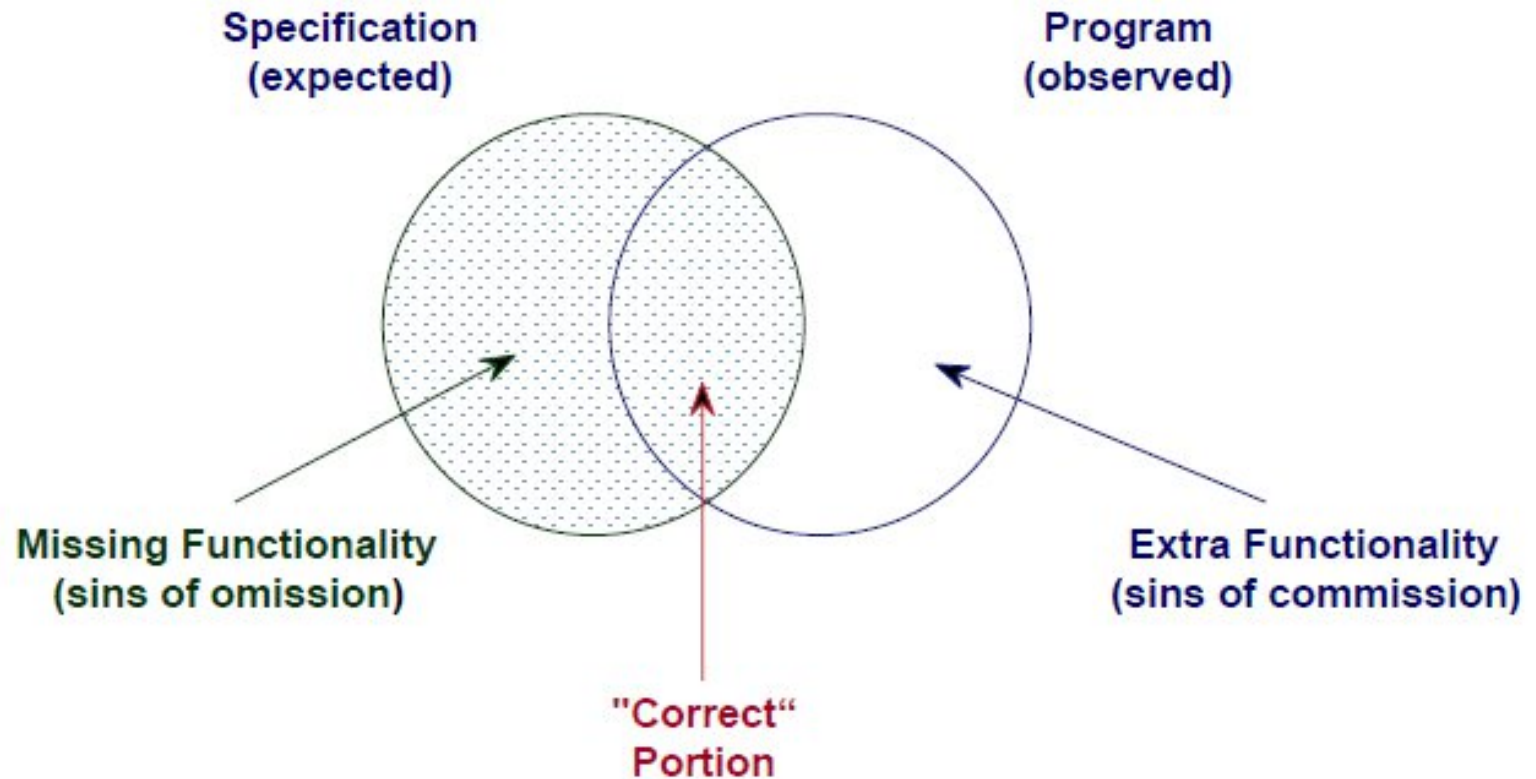
Remarks This is a sample test case.

Created By John Doe

Date of Creation 01/14/2020

Executed By Jane Roe

Insights from a Venn Diagram:



💡 Consider set S of specified behavior and set P of programmed behavior.



- 💡 If certain specified behaviors have not been programmed then they are faults of omission.
- 💡 If certain programmed behaviors have not been specified then they are faults of commission.
- 💡 The intersection of S and P is the correct portion, that is, behaviors that are both specified and implemented.

Identifying Test Cases

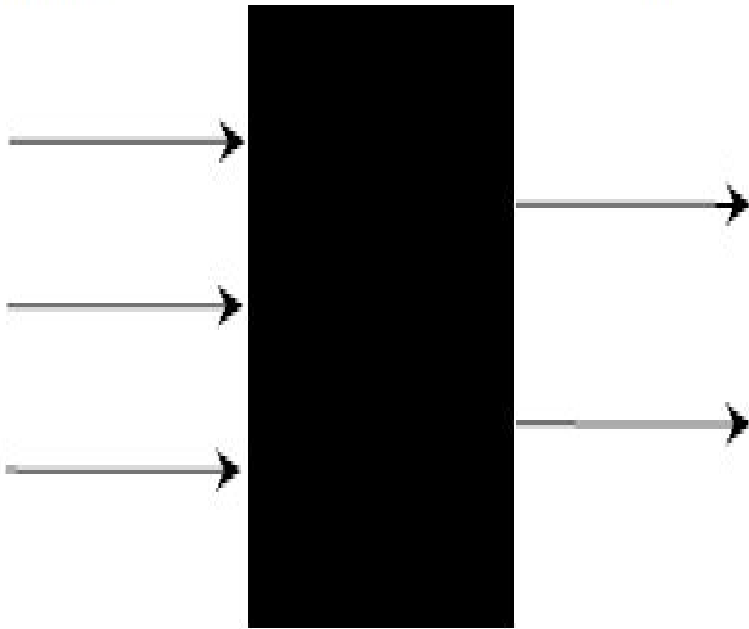


- 💡 Two fundamental approaches are used to identify test cases
 - ✓ Functional Testing
 - ✓ Structural Testing

Functional Testing:

Inputs

Outputs



- Functional testing is based on the view that any program can be considered to be a function that maps values from its input domain to values in its output range.

- This leads to term black box testing, in which the content is not known and function of black box is understood completely in terms of inputs and outputs.

Functional Testing:



- 💡 Functional testing is a type of software testing that focuses on verifying whether the system behaves according to its functional requirements or specifications.
- 💡 In other words, it checks what the system does — not how it does it.

💡 The functional test cases have two distinct advantages:



- ✓ They are independent of the software that is implemented, so if the implementation changes, the test cases are still useful.
- ✓ Test case development can occur in parallel with implementation thereby reducing the overall project development interval.



💡 On the other hand the disadvantages are:

- ✓ Significant redundancies may exist among the test cases.
- ✓ Compounded by the possibility of gaps of untested software.

Structural Testing:



- 💡 This is also known as glass box testing or white box testing.
- 💡 **Structural testing** is where the internal structure, design, and code of the software are tested to ensure that they work as intended.

Structural Testing Techniques:



- 💡 Statement Coverage - This technique is aimed at exercising all programming statements with minimal tests.
- 💡 Branch Coverage - This technique is running a series of tests to ensure that all branches are tested at least once.
- 💡 Path Coverage - This technique corresponds to testing all possible paths which means that each statement and branch are covered.

Error and Fault Taxonomies:



- Our definitions of error and fault hinge on the distinction between process and product: process refers to how we do something , and product is the end result of a process.

Input / Output Faults

Type	Instances
Input	Correct input not accepted
	Incorrect input accepted
	Description wrong or missing
	Parameters wrong or missing
Output	Wrong format
	Wrong result
	Correct result at wrong time (too early, too late)
	Incomplete or missing result
	Spurious result
	Spelling/grammar
	Cosmetic

Logic Faults



- 🧠 Missing case(s)
- 🧠 Duplicate case(s)
- 🧠 Extreme condition neglected
- 🧠 Misinterpretation
- 🧠 Missing condition
- 🧠 Extraneous condition(s)
- 🧠 Test of wrong variable
- 🧠 Incorrect loop iteration
- 🧠 Wrong operator (< instead of <=)

Computation Faults



- 🧠 Incorrect algorithm
- 🧠 Missing computation
- 🧠 Incorrect operand
- 🧠 Incorrect operation
- 🧠 Parenthesis error
- 🧠 Insufficient precision(round-off)
- 🧠 Wrong built-in function

Data Faults



- 🧠 Incorrect initialization
- 🧠 Incorrect storage/access
- 🧠 Wrong flag/index value
- 🧠 Incorrect packing/unpacking
- 🧠 Wrong variable used
- 🧠 Wrong data reference
- 🧠 Incorrect type
- 🧠 Inconsistent data



Interface Faults

Incorrect interrupt handling
I/O timing
Call to wrong procedure
Call to nonexistent procedure
Parameter mismatch (type, number)
Incompatible types
Superfluous inclusion

Levels of Testing



- 💡 A level of software testing is a process where every unit or component of a software/system is tested.
- 💡 There are mainly four testing levels:
 - 💡 1. Unit Testing
 - 💡 2. Integration Testing
 - 💡 3. System Testing
 - 💡 4. Acceptance Testing

1) Unit testing:



- 🧠 A Unit is a smallest testable portion of system.
- 🧠 This kind of testing helps to test each module separately.
- 🧠 The aim is to test each part of the software by separating it.
- 🧠 It checks whether components are fulfilling functionalities or not. This kind of testing is performed by developers.

2) Integration testing:



- Integration means combining.

- In this testing phase, different software modules are combined and tested as a group to make sure that integrated system is ready for system testing.

- Integration testing checks the data flow from one module to other modules. This kind of testing is performed by testers.

3) System testing:



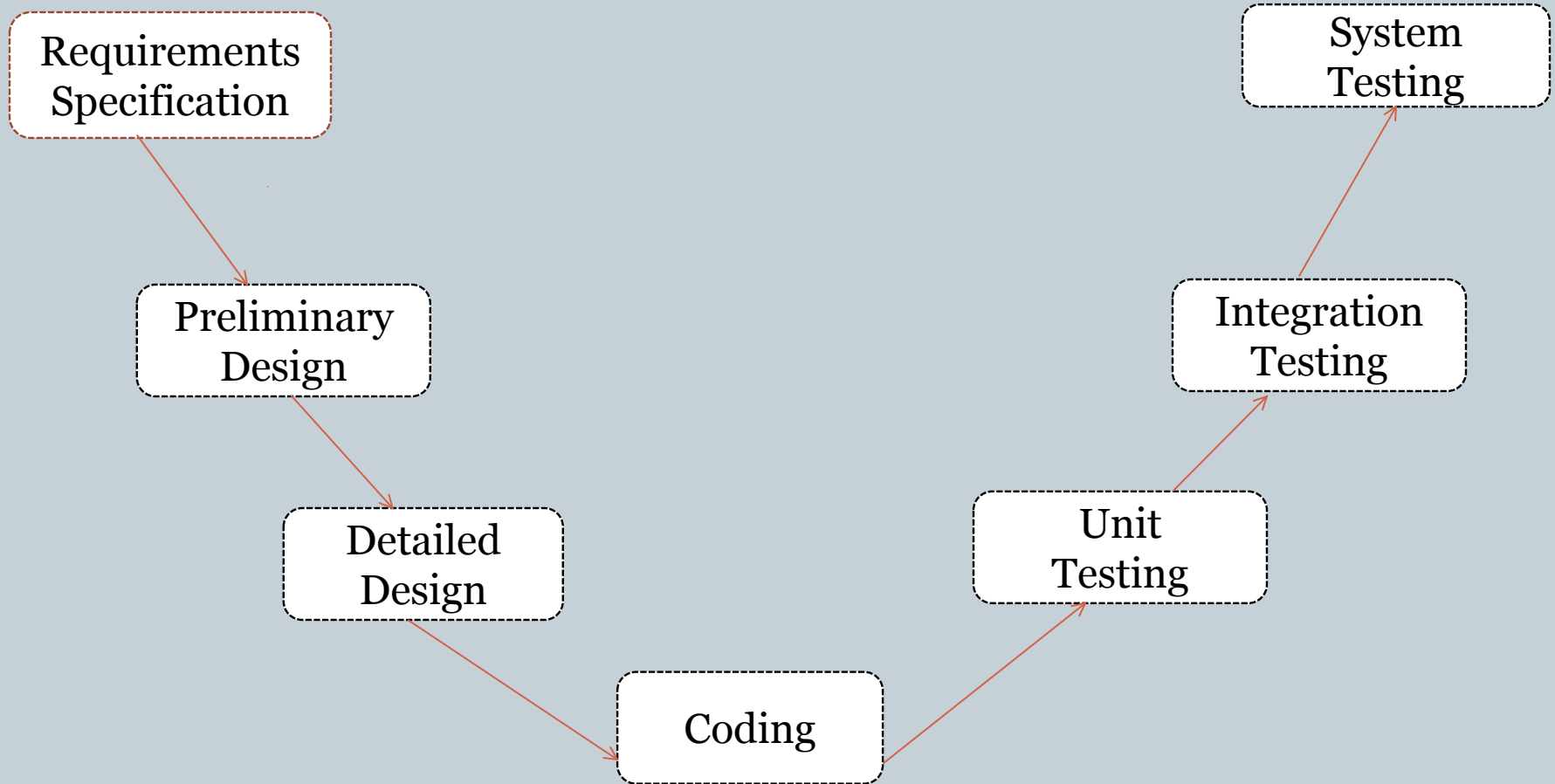
- 💡 System testing is performed on a complete, integrated system.
- 💡 It tests the overall interaction of components.
- 💡 System testing is often the final test to verify that the system meets the specification.
- 💡 It evaluates both functional and non-functional need for the testing.

4) Acceptance testing:



- 💡 Acceptance testing is a test conducted to find if the requirements of a specification or contract are met as per its delivery.
- 💡 Acceptance testing is basically done by the user or customer.
- 💡 However, other stockholders can be involved in this process.

Levels of abstraction and testing in waterfall model.



Software Quality



Quality Attributes

They are divided into 2 types

- Static quality: Refers to the actual code and related documentation
- Dynamic quality: Relates to the behavior of the application while in use



- 💡 Static quality attributes include structured, maintainable, and testable code as well as the availability of correct and complete documentation.
- 💡 Dynamic quality attributes include software reliability, correctness, completeness, usability, consistency and performance
- 💡 **Reliability** refers to the probability of failure- free operation
- 💡 **Correctness** refers to the correct operation of an application



- 💡 **Completeness** refers to the availability of all features listed in the requirements or in the user manual. An incomplete software is one that does not fully implement all features required.
- 💡 **Consistency** refers to a common set of conventions and assumptions.
- 💡 **Usability** refers to the testing of a product by its potential users.



- 💡 **Performance** refers to the time the application takes to perform a requested task.
- 💡 It is considered as a non-functional requirement.

REQUIREMENTS, BEHAVIOR AND CORRECTNESS



- 🧠 Requirements specify what functions the product/software should perform.
- 🧠 Requirements leading to two different programs:
- 🧠 **Requirement 1:** It is required to write a program that inputs two integers and outputs the maximum of these.
- 🧠 **Requirement 2:** It is required to write a program that inputs a sequence of integers and outputs the sorted version of this sequence.

Requirement Incompleteness

- 💡 Suppose that program **max** is developed to satisfy Requirement 1. The expected output of **max** when the input integers are 13 and 19 can be easily determined to be 19.
- 💡 Suppose now that the tester wants to know if the two integers are to be input to the program on one line followed by a carriage return, or on two separate lines with a carriage return typed in after each number. The requirement as stated above fails to provide an answer to this question.

Requirement Ambiguity



- 🧠 Requirement 2 is ambiguous. It is not clear whether the input sequence is to be sorted in ascending or in descending order.
- 🧠 The behavior of **sort** program, written to satisfy this requirement, will depend on the decision taken by the programmer while writing sort.

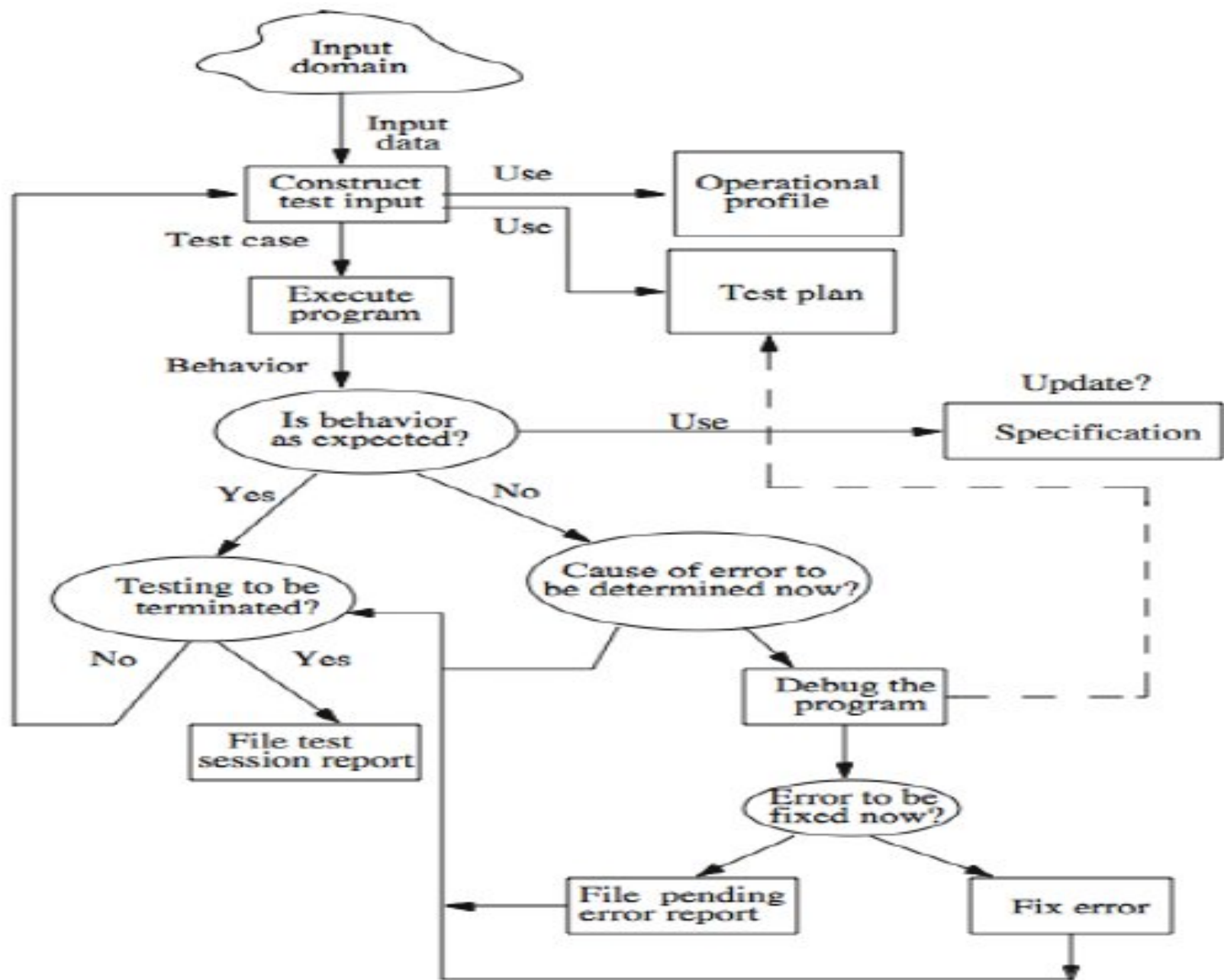
Reliability



- 💡 The reliability of program P is the probability of its successful execution on a randomly selected element from its input domain.
- 💡 Reliability is a continuous metric over a scale from 0 to 1.

Testing and Debugging

- 💡 Testing is the process of determining if a program behaves as expected.
- 💡 When testing reveals an error, the process used to determine the cause of this error and to remove it, is known as debugging.
- 💡 Testing and Debugging are related activities in a cyclic manner.
- 💡 A test and debug cycle is:



1. Preparing a Test Plan



- 🔧 A test cycle is often guided by a test plan
- 🔧 Sample test plan for testing the sort program is shown below.
- 🔧 The sort program is to be tested to meet the requirements given in example.
- 🔧 Execute the program on at least 2 input sequences, one with 'A' and other with 'D'.
- 🔧 Execute the program on an empty input sequence.
- 🔧 Test the program against errors.
- 🔧 All failures of tested program should be recorded in a file.

2. Constructing Test Data



- 💡 A **test case** is a pair consisting of test data to be input to the program and the expected output.
- 💡 The **test data** is a set of values, one for each input variable
- 💡 A **test set** is a collection of zero or more test cases.
- 💡 Program requirements and test plan help in construction of test data.

Test case 1:

Test data: <"A" 12 -29 32 >

Expected output: -29 12 32

Test case 2:

Test data: <"D" 12 -29 32.>

Expected output: 32 12 -29

Test case 3:

Test data: <"A".>

Expected output: No input to be sorted in ascending order

Test case 4:

Test data: <"D".>

Expected output: No input to be sorted in descending order

Test case 5:

Test data: <"R" 3 17.>

Expected output: Invalid request character;

valid characters: "A" and "D"

Test case 6:

Test data: <"A" c 17.>

Expected output: *Invalid number*

3. Executing the program

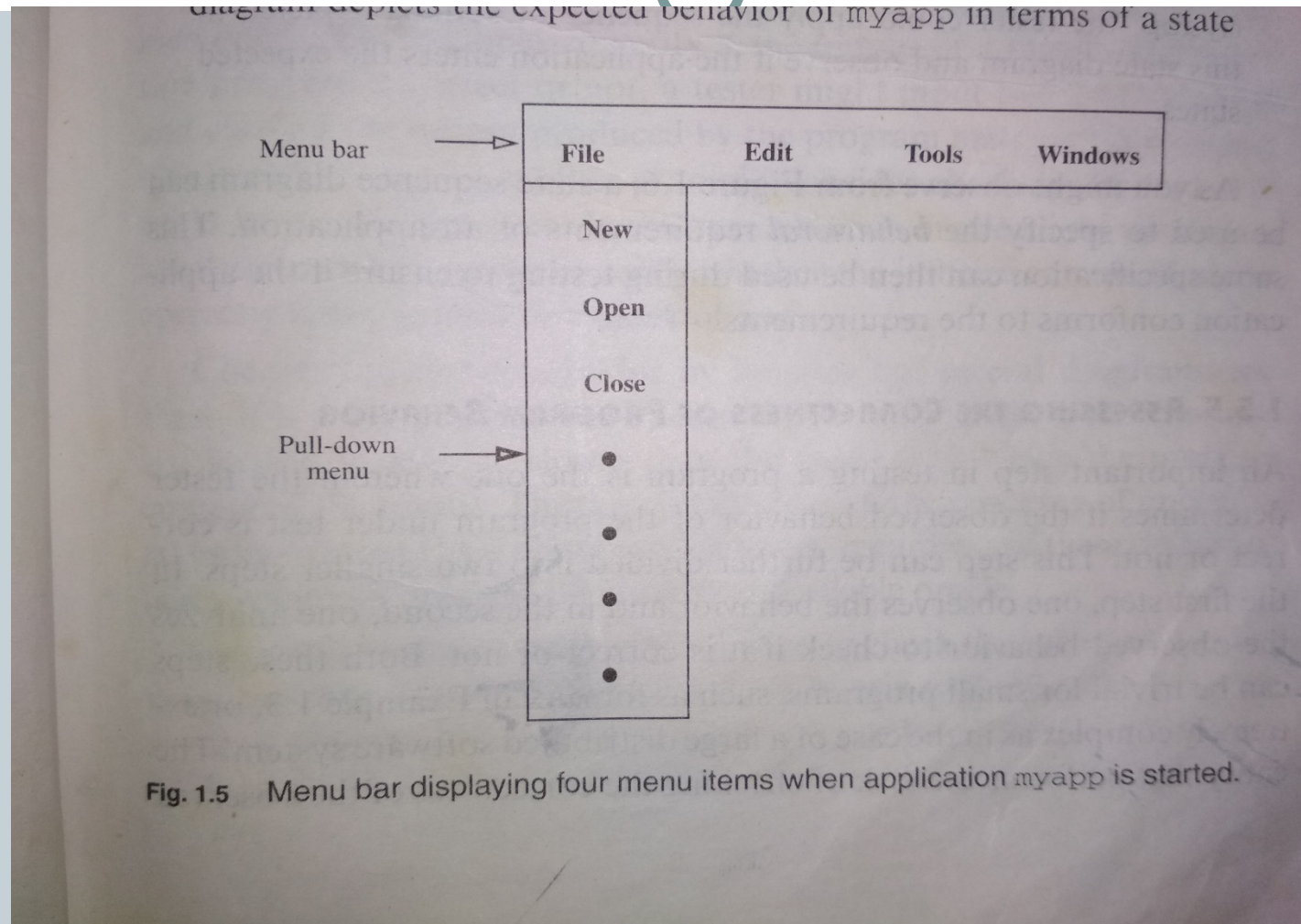


- 💡 A tester might be able to construct a test harness to aid in program execution.
- 💡 A **test harness** is a collection of tools, scripts, and code used to **automate the testing** of software applications.
- 💡 The harness initializes any global variables, inputs a test case, and executes the program.
- 💡 The output generated by the program may be saved in a file for subsequent examination by a tester

4. Specifying program behaviour



- 💡 Program behavior is specified using state diagrams or transition diagrams.
- 💡 Program state can be determined by collecting the current values of program variables into a vector known as the state vector.
- 💡 State sequence diagram can be used to specify the behavioral requirements.
- 💡 This same specification can then be used during the testing to ensure if the application confirms to the requirements



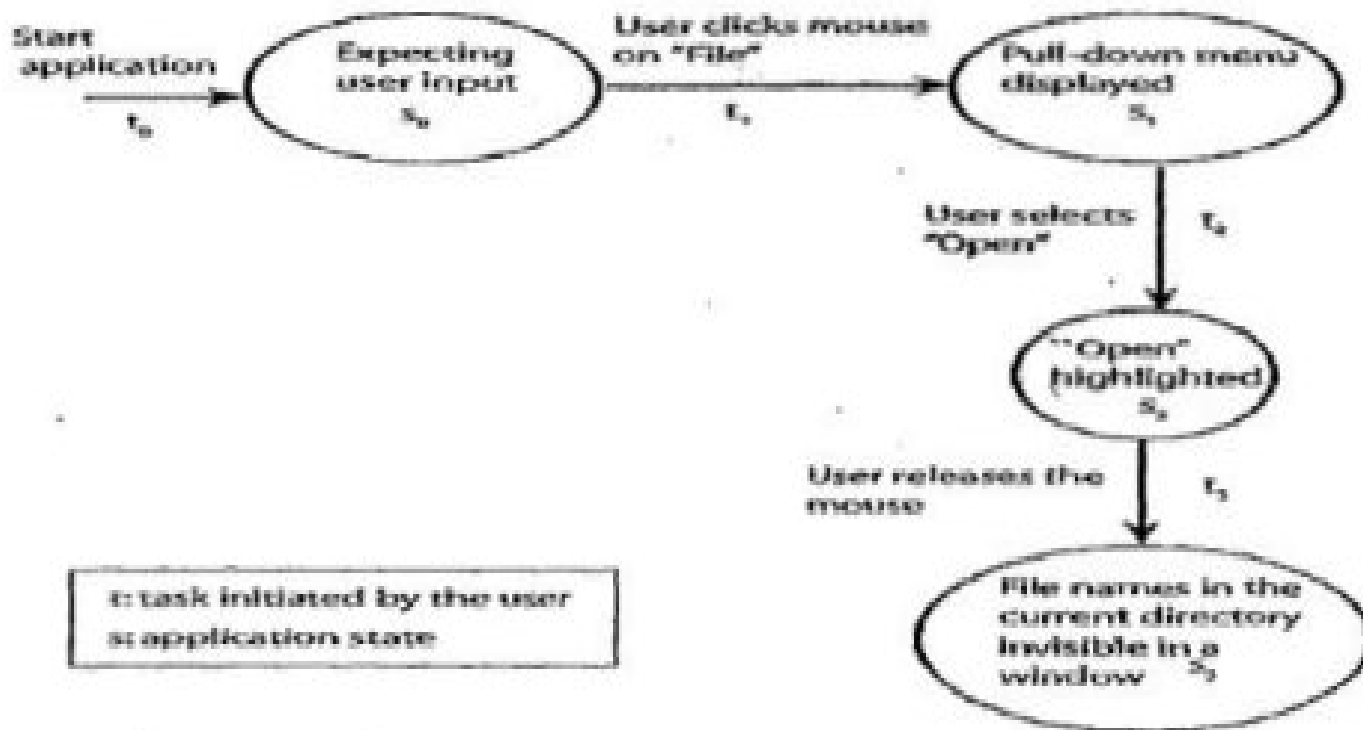


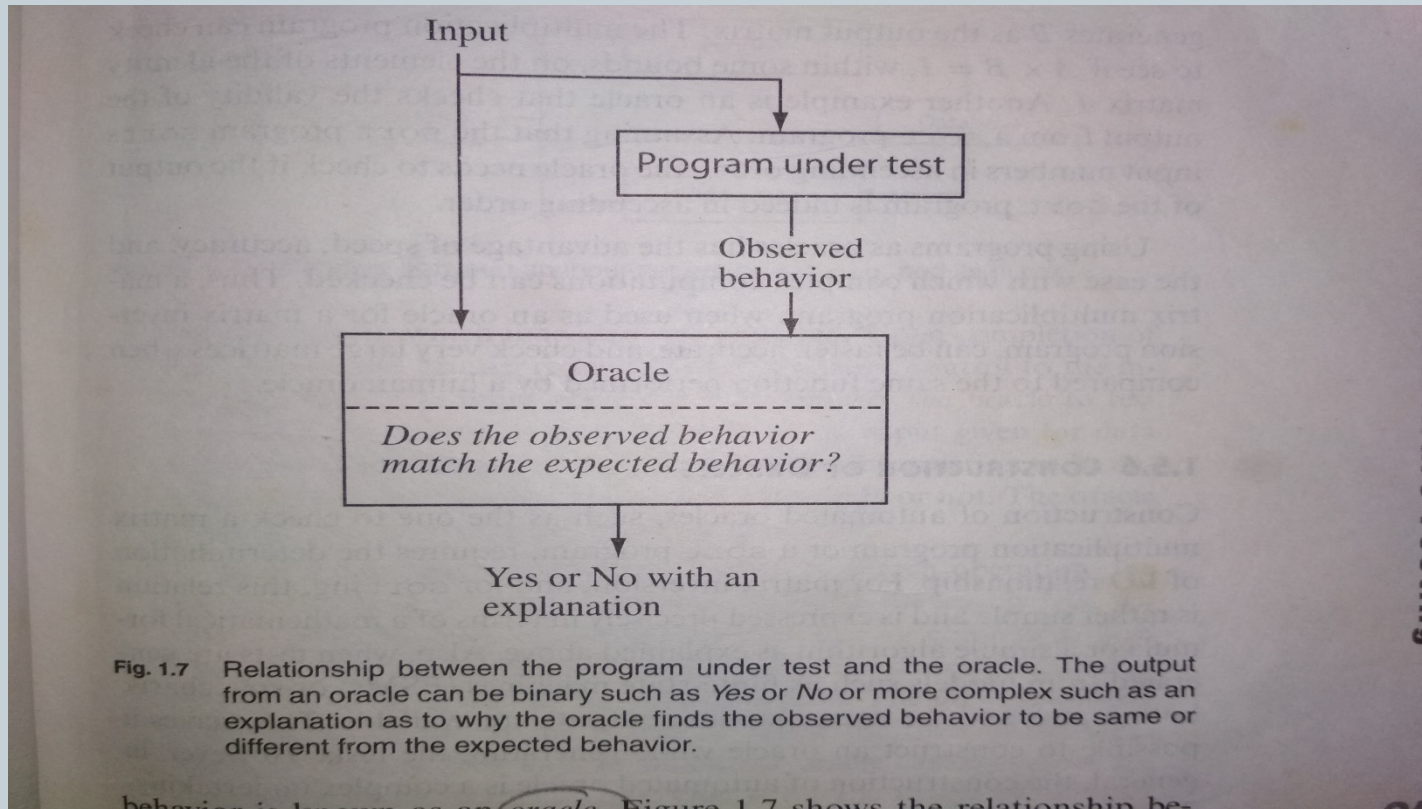
Figure: A state sequence for *myapp* showing how the application is expected to behave when the user selects the `open` option under the `file` menu

5. Assessing the correctness of program behaviour



- 🧠 Tester observes the behaviour of program under test.
- 🧠 Analyzes the observed behaviour to check if it is correct or not.
- 🧠 The entity that performs the task of checking the correctness of the observed behaviour is known as the **oracle**.
- 🧠 **For ex- If a program calculates $2 + 2$, the oracle would check whether the result is 4**

Relationship between program under test and the oracle



Test metrics



💡 The term metric refers to standard of measurement by which efficiency, performance, progress can be measured.

💡 Different types of Test metrics are:

💡 Organizational metric

💡 Project metric

💡 Process metric

💡 Product metric

Organizational metrics



- 💡 Organizational metrics are the metrics at the organization level.
- 💡 Useful in overall project planning and management.
- 💡 Example: Number of defects reported after product release.
- 💡 This type of metrics allows senior management to monitor overall organizational strength.

Project metrics



- 💡 Project metrics relate to a specific project.
- 💡 Useful in monitoring and controlling the project.
- 💡 Example: $\text{Actual test effort} / \text{Planned test effort}$.
- 💡 $\text{Number of successful tests} / \text{Total number of tests}$.

Process metrics



- 💡 Process metric's goal is to evaluate the goodness of the process.
- 💡 When a test process consists of several phases , for example unit test, integration test and system test, one can measure the number of defects in each phase.

Product metrics



- 💡 Product metrics is related specific product.
- 💡 Example- compiler for a programming language.
- 💡 Helpful in making decisions such as 'Should this product be released for use by customer?'

A sample of product metrics:



Metric	Meaning
Reliability	Probability of failure of software product
Defect density	Number of defects

Static and Dynamic metrics



- 💡 **Static: Metrics computed without executing program.**
- 💡 **Ex- Average no. of testers working on a project.**
- 💡 **Dynamic: Metrics require code execution.**
- 💡 **Ex- Number of defects remaining to be fixed.**

Testing and Verification



- 💡 Program verification aims at providing correctness of programs by showing that it contains no errors.
- 💡 Ex- In development of critical applications such as smart cards, or control of nuclear plants.

Test generation strategies

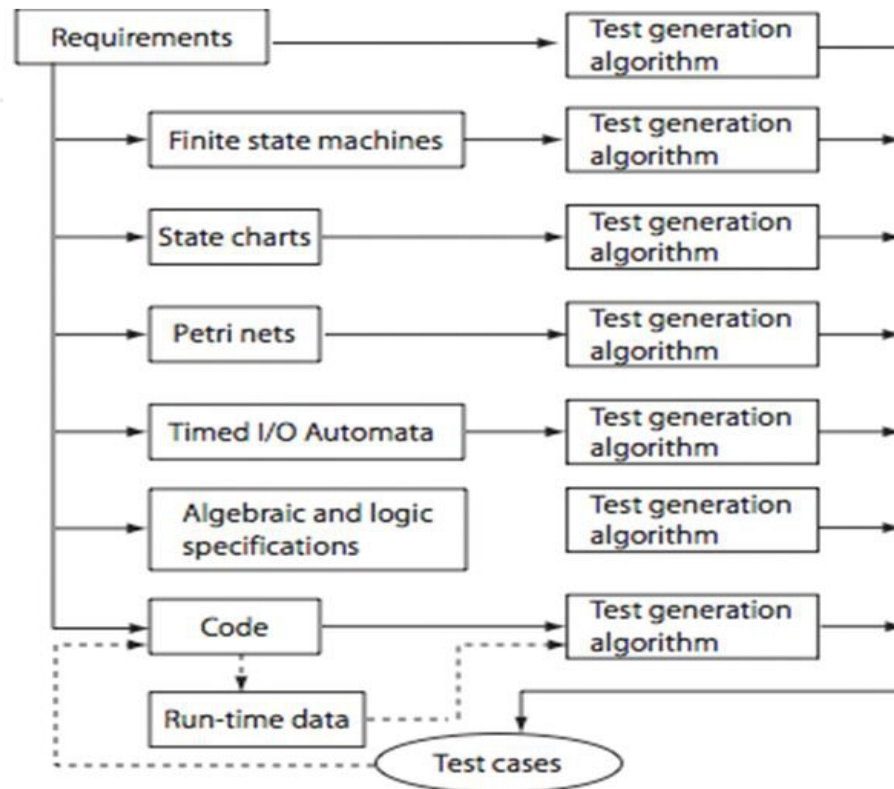


- 💡 One of the key tasks in any software test activity is the generation of test cases.
- 💡 The program under test is executed against the test cases to determine whether it satisfies client's requirements.

Test generation strategies



Test generation strategies (Summary)



Test generation strategies



1. Requirement based test generation.
2. Model based test generation requires a subset of requirements. Ex- Finite state machines, state charts, Petri nets, I/O automata.
3. Algebraic and Logic based test generation.
4. Code based test generation.

Static testing



- 💡 Static testing is carried out without executing the application under test.
- 💡 Carried out by any individual who did not write the code.
- 💡 The test team has to access all required documents, applications, software, user manuals, reports.
- 💡 Also access one or more static testing tools. Tool takes code as input and generates test cases. Ex-Purify from IBM

Walkthroughs



- 🧠 Process to review any application related document.
- 🧠 Requirements are reviewed using "requirements walkthrough" process.
- 🧠 Code is reviewed using "code walkthrough" process.

Inspections



- 🧠 Code is inspected to improve its quality at a lower cost.
- 🧠 Code inspection is carried out by a team according to an inspection plan:
- 🧠 Purpose; software product to be inspected; team formation; tasks to be performed; rate at which inspection is completed;